Bioc Technical Advisory Board Minutes

1 October 2026

Present: Henrik Bengtsson, Lori Kern, Gabriele Sales, Levi Waldron, Ludwig Geistlinger, Rafael Irizarry, Alexandru Mahmoud, Robert Castelo, Vincent Carey, Jacque Serizay, Helena Crowell, Sean Davis, Nicholas Cooley, Andres Wokaty, Michael Lawrence, Wolfgang Huber, Hugo Gruson, Hervé Pagès, Kasper D. Hansen
Absent: Laurent Gatto

:05 September minutes approved.

:06 **TAB election** results: current board remains unchanged.

:07 - :39 **Sean Davis** on Bioconductor infrastructure.

- Infrastructure project website
- Web and packages infrastructure slides
- Bioconductor by the numbers dashboard
  (number of packages, publications, citations, etc.)

:40 - :44 Reducing “shade” on **R’s performance**.

- Should we introduce a warning when it is discovered
  that a non-optimized blas is in use? As part of BiocManager?
- Philosophy of “users becoming developers”
  might be another reason for “bad” performance.

:45 - :57 **Agentic review** of “incoming” Bioconductor packages.

- Background and Example at BiocPkgReviewer (private GitHub repo)
- Example of a triage document for VariantAnnotation (with many open issues and PRs)
- github.com/bioconductor/ai-agent-skills intended for authors to use on their own
- For your packages, use the action noted here: https://bioconductor.github.io/workflows/

:58 Future of **support site**: Will likely be archived > Move to Discourse.

Appendix:

Google search result: **Apple's vecLib** (part of the Accelerate framework) is highly optimized for Mac hardware, but scientific and statistical communities (most notably **CRAN/R** and **NumPy/SciPy**) have historically flagged precision discrepancies compared to standard, reference BLAS implementations (like Netlib BLAS or OpenBLAS). [1, 2, 3]

The inaccuracies and inconsistencies observed with vecLib
generally stem from the following technical behaviors:

1. Non-Associative Accumulation from Parallelization

Standard BLAS architectures often process operations linearly or through predictable loops. To maximize performance on Apple Silicon or Intel Macs, vecLib aggressively parallelizes vector instructions across SIMD registers or custom coprocessors (like Apple's AMX blocks). [1, 2, 3]

- Because floating-point addition is **not associative** ((a + b) + c ≠ a + (b + c) in IEEE 754 arithmetic), changing the order in which a massive stream of numbers is summed alters the final rounding bits. [1]
- For very large matrices, this can cause the tailing decimal places of dot products or matrix multiplications to drift away from reference BLAS results. [1, 2]

2. Aggressive Use of Fused Multiply-Add (FMA)

vecLib heavily relies on hardware-level **Fused Multiply-Add (FMA)** instructions. [1]

- An FMA computes a × b + c with only a **single rounding step** at the end, rather than rounding once after the multiplication and again after the addition.
- While FMA is technically more accurate than un-fused operations, it produces distinctly different bitwise results than standard reference implementations that compute multiplication and addition sequentially. For chaotic or poorly conditioned algorithms, this slight divergence can cascade into noticeably different final outputs.

3. Edge-Case Handlings and Historical LAPACK Bugs

The main reason major scientific packages like NumPy and SciPy historically dropped or restricted default support for Apple Accelerate/vecLib was due to specific bugs in the higher-level LAPACK routines mapped over the BLAS layer. [1]

- **Complex Number Edge Cases:** Older versions of vecLib occasionally mishandled edge-case inputs (such as NaNs, Infinities, or underflows) in complex-valued matrix factorizations (like SVD or Eigendecomposition), sometimes returning incorrect signs or failing to converge where reference libraries succeeded.
- **Outdated Codebases:** For a long period, Apple's underlying LAPACK layer was frozen at a much older reference version (LAPACK 3.2.1) while the rest of the open-source ecosystem moved forward, introducing silent compatibility errors. (Note: Apple has since modernised this, upgrading to LAPACK 3.12.0 in recent OS versions). [1]

Comparison Summary

Metric / Behavior	Reference BLAS (Netlib / R Built-in)	Apple vecLib / Accelerate
Primary Goal	Bitwise reproducibility and standard compliance.	Peak hardware execution and speed.
Summation Order	Strict, predictable loops.	Dynamic, hardware-dependent parallel blocks.
FMA Utilization	Depends entirely on compiler flags.	Deeply integrated into the underlying framework.

1: https://developer.apple.com/documentation/accelerate/blas-library
2: https://ulrichatz.org/blog/2026-06-04-r-blas-macos/
3: https://news.ycombinator.com/item?id=25062479
(archived discussion at scipy: https://github.com/scipy/archive/blob/main/wiki/dropping-accelerate-support.md)
https://github.com/scipy/scipy/issues/20893 indicates scipy does support linking accelerate
https://numpy.org/doc/2.2/building/index.html numpy can also use accelerate